

Implementing Toxic Language Detection with Guardrails for Google Gemini

Introduction

When building applications with Google Gemini, ensuring appropriate content generation is crucial. This guide explains how to implement toxic language detection using the Guardrails framework, which helps filter out potentially harmful or offensive content.

Setting Up Your Environment

Let's start by setting up the necessary packages and components:

Install the specific version

pip install guardrails-ai==0.3.0

Import required libraries

import google.generativeai **as** genai *# For accessing Google Gemini*

import guardrails **as** gd *# The safety framework*

from guardrails.hub **import** ToxicLanguage *# Specific validator for toxic language*

Configure Gemini API

genai.configure(api_key="YOUR_GEMINI_API_KEY")

Initialize Gemini model

model = genai.GenerativeModel('gemini-1.5-pro')

Create a Guard with toxic language detection

```
guard = gd.Guard().use(  
    ToxicLanguage(on_fail="fix")  
)
```

Code Explanation: - We import three essential components: Google's Gemini API, Guardrails framework, and the ToxicLanguage validator - We configure Gemini with an API key (you'll need to get this from Google) - We initialize a Gemini model instance (specifically gemini-1.5-pro) - We create a Guard object with the ToxicLanguage validator, setting it to fix toxic content when detected

The last line is particularly important: - `gd.Guard()` creates a new Guard instance - `.use(ToxicLanguage(on_fail="fix"))` adds the toxic language validator with instructions to fix (rewrite) toxic content - By default, it uses sentence-level validation with a threshold of 0.25 (moderate sensitivity)

Basic Implementation: Ensuring Safe Responses

Here's how to create a simple function that generates safe content with Gemini:

```
def generate_safe_response(prompt):
    # Generate content with Gemini
    response = model.generate_content(prompt)
    raw_response = response.text

    # Check and fix any toxic language
    raw_llm_output, validated_output, *rest = guard.parse(
        llm_output=raw_response,
    )

    # Return the safe version
    return validated_output

# Example usage
safe_response = generate_safe_response("Write about poor customer service")
print(safe_response)
```

Code Explanation: 1. Our function takes a user prompt as input 2. It sends the prompt to Gemini and gets the raw response 3. It passes that response through our ToxicLanguage guard 4. The guard.parse() function returns multiple values: - raw_llm_output: The original unmodified text - validated_output: The safe version with toxic content fixed - *rest: Additional validation information (which we don't need here) 5. We return only the validated (safe) output

Example Output:

I recently experienced disappointing customer service at a local store. The staff appeared disinterested and took an extended time to acknowledge my presence. When I asked for assistance finding a product, the employee gave vague directions without offering to help directly. My questions about product features were met with minimal information, and I had to seek out details independently. When I finally made a purchase, the checkout process was slow with no apology for the wait. Basic courtesies like eye contact and thanking me for my business were absent. This experience left me unlikely to return as a customer.

Notice how the response is critical of the service but maintains respectful language.

Understanding Toxic Language Detection

Let's see how the validator works with an example containing mixed content:

```
# Example text with both appropriate and inappropriate content
mixed_text = """The feature works well overall and performs as expected.
Your coding skills are terrible, this is the worst implementation I've ever
seen."""

# Check and fix toxic content
raw_llm_output, validated_output, *rest = guard.parse(
    llm_output=mixed_text,
```

```
)

print("Original text:")
print(mixed_text)
print("\nSafe version:")
print(validated_output)
```

Code Explanation: - We create sample text containing both appropriate and toxic content - We run it through our guard to detect and fix toxic language - We print both versions to compare the changes

Expected Output:

Original text:

The feature works well overall and performs as expected.
Your coding skills are terrible, this is the worst implementation I've ever seen.

Safe version:

The feature works well overall and performs as expected.
Your coding approach could be improved, as this implementation has several areas that need enhancement.

Notice how: 1. The first sentence remains untouched because it contains no toxic language
2. The second sentence is rewritten to provide the same feedback but in a constructive, non-toxic way
3. The guard identified and fixed only the problematic part while preserving the rest

Configuration Options

Validation Methods: Sentence vs. Full Text

The ToxicLanguage validator offers two approaches to checking content:

```
# Create a guard that checks entire text as one unit
full_guard = gd.Guard().use(
    ToxicLanguage(validation_method="full", on_fail="fix")
)

# Check highly toxic content
toxic_message = "Stop being such a dumb piece of shit. Why can't you comprehend this?"
_, full_validated, *_ = full_guard.parse(llm_output=toxic_message)

# Check with default sentence-level validation for comparison
_, sentence_validated, *_ = guard.parse(llm_output=toxic_message)

print("Original:", toxic_message)
print("Full-text validation:", full_validated)
print("Sentence validation:", sentence_validated)
```

Code Explanation: - We create a new guard with `validation_method="full"` which evaluates the entire text as a single unit - We compare this to our original guard that uses the default sentence-by-sentence checking - Both guard configurations use `on_fail="fix"` to rewrite toxic content

Expected Output:

Original: Stop being such a dumb piece of shit. Why can't you comprehend this?

Full-text validation: I need you to try to understand this concept. Would you like me to explain it differently?

Sentence validation: Please try harder to understand this. Why are you having difficulty comprehending this?

The difference is important: - **Sentence validation** (default): Checks and fixes each sentence individually - **Full-text validation**: Evaluates the entire content as a whole, often resulting in more complete rewrites

Customizing Sensitivity with Thresholds

You can adjust how sensitive the toxic language detection is:

```
# Different sensitivity levels
strict_guard = gd.Guard().use(ToxicLanguage(threshold=0.1, on_fail="fix"))
# Very sensitive
standard_guard = gd.Guard().use(ToxicLanguage(threshold=0.25, on_fail="fix"))
# Default
relaxed_guard = gd.Guard().use(ToxicLanguage(threshold=0.6, on_fail="fix"))
# Less sensitive

# Test borderline content
borderline_text = "This idea is stupid, but let me explain a better approach."

# Check with each sensitivity level
_, strict_result, *_ = strict_guard.parse(llm_output=borderline_text)
_, standard_result, *_ = standard_guard.parse(llm_output=borderline_text)
_, relaxed_result, *_ = relaxed_guard.parse(llm_output=borderline_text)

print("Original:", borderline_text)
print("Strict:", strict_result)
print("Standard:", standard_result)
print("Relaxed:", relaxed_result)
```

Code Explanation: - We create three guards with different thresholds: - 0.1: Low threshold means high sensitivity (catches more potential issues) - 0.25: Default threshold, moderate sensitivity - 0.6: High threshold means low sensitivity (only catches clearly toxic content) - We test each against the same borderline example to see the differences

Expected Output:

Original: This idea is stupid, but let me explain a better approach.
Strict: This idea is flawed, but let me explain a better approach.
Standard: This idea is flawed, but let me explain a better approach.
Relaxed: This idea is stupid, but let me explain a better approach.

The strict and standard guards catch and fix the word “stupid,” while the relaxed guard lets it pass.

Handling Toxic Content: Different Approaches

The ToxicLanguage validator offers several ways to handle toxic content:

```
# Different handling strategies
fix_guard = gd.Guard().use(ToxicLanguage(on_fail="fix"))           # Rewrite
toxic_content
filter_guard = gd.Guard().use(ToxicLanguage(on_fail="filter"))    # Remove
toxic_content
refrain_guard = gd.Guard().use(ToxicLanguage(on_fail="refrain")) # Replace
with standard message

# Test with toxic content
toxic_text = "This product is absolute garbage and only idiots would buy it."

# Apply different strategies
_, fix_result, *_ = fix_guard.parse(llm_output=toxic_text)
_, filter_result, *_ = filter_guard.parse(llm_output=toxic_text)
_, refrain_result, *_ = refrain_guard.parse(llm_output=toxic_text)

print("Original:", toxic_text)
print("Fix:", fix_result)
print("Filter:", filter_result)
print("Refrain:", refrain_result)
```

Code Explanation: - We create three guards with different on_fail behaviors: - "fix": Attempts to rewrite toxic content to be non-toxic - "filter": Completely removes toxic content - "refrain": Replaces toxic content with a standard message - We test each approach on the same toxic text sample

Expected Output:

Original: This product is absolute garbage and only idiots would buy it.
Fix: This product is of poor quality and I wouldn't recommend it.
Filter: This product is bad.
Refrain: I apologize, but I cannot provide the response as it contains content that may be inappropriate.

Each approach handles the toxic content differently: - fix: Rewrites the content while preserving the general meaning - filter: Removes the toxic parts, potentially leaving an incomplete message - refrain: Replaces the entire message with a standard refusal message

Complete Implementation Example

Now let's put everything together in a more robust implementation:

```
def safe_gemini_chat(prompt, safety_level="standard"):
    """Generate safe responses from Gemini with configurable safety levels"""

    # Select appropriate guard based on safety level
    if safety_level == "strict":
        current_guard = gd.Guard().use(ToxicLanguage(threshold=0.1,
on_fail="fix"))
    elif safety_level == "relaxed":
        current_guard = gd.Guard().use(ToxicLanguage(threshold=0.5,
on_fail="fix"))
    else: # standard
        current_guard = gd.Guard().use(ToxicLanguage(threshold=0.25,
on_fail="fix"))

    # First, check if the user prompt itself is toxic
    try:
        _, checked_prompt, *_ = current_guard.parse(llm_output=prompt)

        # If prompt was substantially changed, it was likely toxic
        if len(checked_prompt) < len(prompt) * 0.7:
            return "I'm sorry, but I can't respond to that type of question."
    except Exception as e:
        return f"Error checking prompt: {str(e)}"

    # Generate content with Gemini
    try:
        response = model.generate_content(checked_prompt)
        raw_response = response.text
    except Exception as e:
        return f"Error generating response: {str(e)}"

    # Check Gemini's response for toxic content
    try:
        _, safe_response, *_ = current_guard.parse(llm_output=raw_response)
        return safe_response
    except Exception as e:
        return f"Error validating response: {str(e)}"
```

Code Explanation: 1. Our function takes two parameters: - prompt: The user's question or instruction - safety_level: How strict to be with the toxic content detection (defaulting to "standard")

2. Based on the safety level, we create a guard with the appropriate threshold:
 - "strict": 0.1 threshold (catches more potentially toxic content)
 - "standard": 0.25 threshold (default)

- “relaxed”: 0.5 threshold (only catches clearly toxic content)
- 3. We run a two-way check:
 - First, we check if the user’s prompt itself contains toxic content
 - Then, we generate content with Gemini and check if its response contains toxic content
- 4. We have robust error handling at each step:
 - If checking the prompt fails, we return an error
 - If generating the Gemini response fails, we return an error
 - If validating the response fails, we return an error
- 5. The function returns only validated, safe content to the user

This implementation provides comprehensive protection by: - Checking both input and output for toxic content - Adapting to different safety needs with configurable levels - Handling errors gracefully at every step of the process

Best Practices & Testing

Best Practices

- **Handle API Errors:** Implement robust error handling as shown in the complete example.
- **Optimize Input Size:** Keep prompts concise to improve processing speed and stay within API limits.
- **Iterate and Test:** Regularly test with different inputs to ensure your toxic language detection works effectively.

Common Pitfalls

- **Ignoring Documentation:** Always refer to Google Gemini’s API documentation to understand limitations.
- **Overlooking Rate Limits:** Be mindful of API usage guidelines, especially in high-traffic applications.

Performance Evaluation

When testing your implementation, consider these key metrics:

Metric	Description
Accuracy	Ratio of correctly identified instances over total instances
Precision	Ratio of true positives to the sum of true positives and false positives
Recall	Ratio of true positives to the sum of true positives and false negatives
F1 Score	Harmonic mean of precision and recall

Test Cases to Include

- **Neutral Content:** Normal, non-toxic text to ensure it passes through unchanged
- **Clearly Toxic Content:** Obvious examples of toxic language

- **Borderline Content:** Text that's on the edge of what might be considered toxic
- **Context-Dependent Phrases:** Content that might be toxic in some contexts but not others

Conclusion

Implementing toxic language detection with Guardrails for Google Gemini helps ensure your AI applications generate safe, appropriate content. By configuring the ToxicLanguage validator properly, you can:

1. Detect potentially harmful content in both user inputs and AI outputs
2. Customize sensitivity levels for different use cases
3. Handle toxic content through various approaches (fixing, filtering, etc.)
4. Build reliable safety mechanisms into your AI applications

While no safety system is perfect, these guardrails provide an effective first line of defense against toxic content generation.